

OpenSSL in 4D using System Workers

By Al Mahdi Bakkali, Technical Support Engineer, 4D Inc.

Technical Note 26-02

Table of Contents

Table of Contents	2
Abstract.....	3
Introduction	3
System Requirements	3
OpenSSL Installation	4
Windows Installation	4
MacOS Installation	4
Why OpenSSL in 4D?	5
Asynchronous Architecture.....	5
SystemWorker Class Pattern	5
The Execute_OpenSSLAsynch Method	6
The onFileGenerated Callback	7
Automated OpenSSL Configuration.....	8
The Configuration File Structure	8
Cryptographic Resource Generation.....	8
Private Key Generation	9
Certificate Signing-Request Generation	10
Command Parameters Explained.....	10
Self-Signed Certificate Generation	10
Signing XML file Generation	11
4D Web Server Integration.....	12
Best Practices & Security Considerations.....	13
Certificate Management.....	13
Production Deployment	13
Troubleshooting Common Issues	14
OpenSSL Not Found.....	14
Invalid XML Signature	14
Conclusion	14

Abstract

This document outlines the integration of OpenSSL within a 4D application environment to generate cryptographic assets using asynchronous execution patterns. This technical note leverages the 4D.SystemWorker class to enable non-blocking shell executions for generating Private Keys, Certificate Signing Requests (CSR), and self-signed certificates. The implementation demonstrates practical use cases including the production of signed XML documents for secure data exchange and the deployment of a 4D Web server configured with generated certificates. By adopting an asynchronous callback-based architecture, this integration ensures that cryptographic operations, which can be computationally intensive and time-consuming, do not interrupt the responsiveness of the user interface or block other application processes.

Introduction

In the modern landscape of digital security, the ability to generate, manage, and utilize cryptographic assets such as private keys, Certificate Signing Requests (CSR), and signed certificates and documents are no longer an optional feature; it is a fundamental requirement for compliance, e-invoicing, and secure data exchange. Organizations operating in regulated industries face strict mandates regarding data security and identity verification, making robust cryptographic infrastructure essential for business operations. Furthermore, the increasing prevalence of electronic invoicing systems and supply chain automation has created a need for developers to implement sophisticated digital signature mechanisms that can verify the authenticity and integrity of transmitted data.

This document serves as a comprehensive technical manual for integrating OpenSSL within the 4D development environment using the **4D.SystemWorker** class. By moving away from legacy synchronous execution patterns with **LAUNCH EXTERNAL PROCESS** that block application threads and embracing the asynchronous capabilities of System Workers, developers can build responsive, high-performance applications that interact directly with the operating system's command-line interface. The asynchronous approach represents a significant architectural evolution, allowing cryptographic operations to execute in the background while the main application thread remains available to process user interactions and handle other concurrent tasks. This transition from blocking to non-blocking execution is particularly important for applications that must maintain responsiveness during certificate generation, which can take several seconds or even minutes depending on key size and system resources.

System Requirements

The demonstration 4D application to this technical note ensures cross-platform compatibility and was developed for both Windows and macOS operating systems. To achieve this cross-platform functionality, the system implements dynamic detection of the OpenSSL binary path, automatically locating the executable in the default installation path for each respective platform. This automatic detection significantly reduces configuration complexity for end users while still maintaining flexibility for custom installations. If OpenSSL exists in a non-standard folder location, the application provides a mechanism for users to manually specify the

executable path, which is then stored in the application session and used for all subsequent operations.

This tech note was produced in the following context:

- 4D 21 LTS (<https://us.4d.com/product-download/4D-21-LTS>)
- OpenSSL version 3.0 or later
- Windows 11 – macOS Tahoe 26 (Apple Silicon)

OpenSSL Installation

For developers who do not yet have OpenSSL installed on the development machines, the installation process is straightforward on both supported platforms.

Windows Installation

Windows users can leverage the Windows Package Manager by executing **winget install openssl** from a command prompt or PowerShell window, which automatically downloads and installs the latest stable version of OpenSSL along with all necessary dependencies.

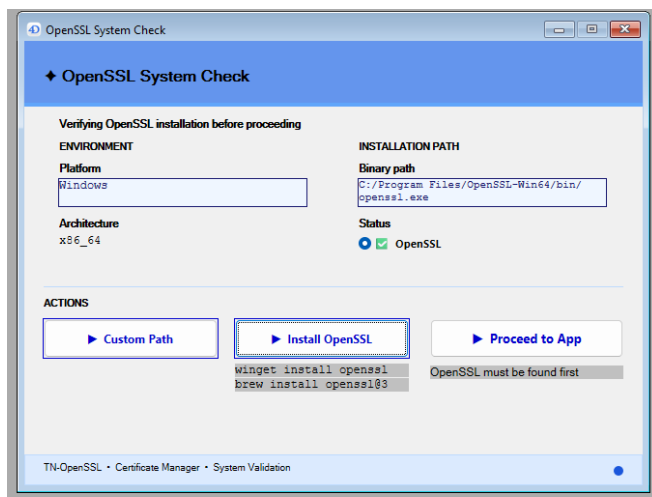
```
winget upgrade
winget install openssl
winget search openssl
```

MacOS Installation

macOS users can utilize Homebrew, the popular package manager for macOS in the Terminal application.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install openssl
```

In the demonstration application, the user can check if OpenSSL was successfully installed thanks to the OpenSSL system check form.



Why OpenSSL in 4D?

While 4D provides built-in commands for many cryptographic tasks, OpenSSL remains the industry standard for complex cryptographic operations. By leveraging 4D.SystemWorker, we gain:

- **Asynchronous Execution:** Asynchronous operations using callback patterns that keep applications responsive. Non-blocking execution allows the UI to remain fluid while heavy encryption tasks run in the background.
- **Flexibility:** Access to the full suite of OpenSSL parameters.
- **Customization:** Combine OpenSSL commands and parameters to meet third-party software requirements.
- **Automation of Identity:** Utilizing custom OpenSSL configuration files (**openssl.cnf**) to automatically add organizational data, such as unique Serial Numbers and Distinguished Names (DN), into certificates without manual user intervention.

Asynchronous Architecture

SystemWorker Class Pattern

The implementation of this OpenSSL integration follows a class-based callback pattern, which is the recommended approach in the 4D SystemWorker documentation. This architectural decision provides significant advantages over alternative approaches. By centralizing all callback logic within a single class, the codebase achieves clean organization that makes the system easier to understand and modify. The class `OpenSSLWorker` structure allows for sophisticated state management using the `This` keyword, enabling data and context to be maintained across multiple callback invocations.

The `OpenSSLWorker` class serves as the central hub for managing all SystemWorker callbacks during OpenSSL command execution. According to the 4D SystemWorker documentation, all callback functions receive two object parameters: the first parameter is always the SystemWorker object itself, providing access to properties like `response`, `responseError`, and `exitCode`, while the second parameter is an event object containing metadata about the callback type and, in some cases, data chunks. The class implements five distinct callback functions that correspond to different stages of the command execution lifecycle.

The `onResponse` function is called when the complete response has been received, though in practice, the `onData` callback is more commonly used for processing incremental output. The `onData` function is invoked each time a chunk of data arrives from the standard output stream, making it ideal for monitoring progress during long-running operations. Similarly, `onDataError` handles data arriving from the standard error stream. The `onError` function captures execution errors that represent unusual runtime conditions, such as the inability to locate the executable or permission issues. Finally, the `onTerminate` function is guaranteed to be called at the end of execution regardless of success or failure, making it the ideal location for cleanup operations and final status determination.

```
Function onResponse($param1 : Object; $param2 : Object)
// $param1 - SystemWorker
// $param2.type - "response"
customLog($param2.type)
```

The class constructor accepts a completion function that will be invoked upon successful execution, allowing each OpenSSL operation to define its own post-execution behavior.

The timeout property is particularly important as it is automatically read by the SystemWorker instance, providing a mechanism to prevent indefinitely hung processes without requiring manual timeout management in the callback functions themselves.

The Execute_OpenSSLAsynch Method

The core execution method represents a sophisticated wrapper around the 4D SystemWorker class that automatically handles the complexities of worker process management. The method begins by detecting whether it is currently executed within a worker process. If not, then a worker is created, which is a critical requirement according to the 4D documentation.

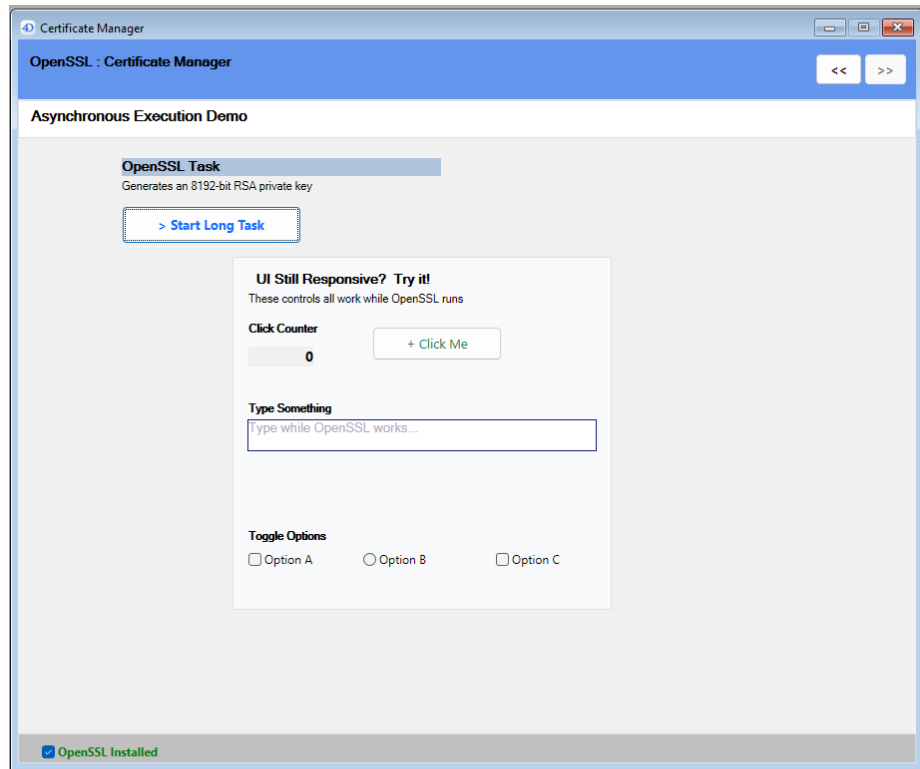
As explicitly stated in the SystemWorker class documentation:

Note: *For the callback functions to be called when [...] do not use wait() (asynchronous call), the process must be a worker created with CALL WORKER, NOT New process.*

```
If (Process info(Current process).type#Worker process)
  CALL WORKER("OpenSSL_Worker"; Current method name; $command; $onComplete)
  return
End if
```

Once executing within a worker, the method validates the OpenSSL installation path from the application's storage configuration and constructs the full command string with proper path quoting to handle spaces in directory names. It then instantiates the OpenSSLWorker class, passing the completion callback, and creates the SystemWorker instance using this class object as the options parameter. Critically, the method does not call the wait() function on the SystemWorker, which would block execution and defeat the purpose of the asynchronous architecture. Instead, it returns immediately, allowing the calling code to continue execution while the OpenSSL command runs in the background, with callbacks firing as events occur during the command's lifecycle.

This architecture delivers true non-blocking execution where the user interface remains responsive throughout potentially lengthy cryptographic operations. The automatic worker detection and self-spawning mechanism makes the asynchronous behavior transparent to calling code, which simply invokes Execute_OpenSSLAsynch with a command and callback function without needing to understand the underlying worker process requirements. This abstraction significantly reduces the complexity burden on developers implementing OpenSSL operations while ensuring adherence to the requirements documented in the 4D SystemWorker class reference.



The onFileGenerated Callback

The `onFileGenerated` callback project method provides a generic, reusable handler for all file generation operations in the cryptographic workflow. Rather than implementing separate completion handlers for private keys, CSRs, certificates, and signed XML files, this unified callback accepts parameters that identify which file was generated and what should be done with it upon successful completion. This design significantly reduces code duplication and ensures consistent post-completion behavior across all OpenSSL operations in the application.

```
Execute_OpenSSLAsync($cmd; Formula(onFileGenerated($keyPath)))
```

The callback converts the file path into a 4D File object, which provides access to the object-oriented file handling including the `platformPath` property. This property is essential for the `SHOW ON DISK` command, which reveals the generated file in the operating system's file browser—Finder on macOS or File Explorer on Windows—and requires a platform-specific path format. On Windows, POSIX-format paths using forward slashes are not understood by `SHOW ON DISK`, making the `platformPath` conversion critical for cross-platform compatibility. By revealing the generated file to the user upon completion, the application provides immediate visual confirmation that the operation succeeded while also giving users easy access to the generated cryptographic assets for inspection or further processing.

Automated OpenSSL Configuration

The foundation of the cryptographic workflow lies in the **openssl.cnf** configuration file, which plays a central role in automating certificate generation and ensuring consistency across operations. Rather than relying on manual input during the CSR (Certificate Signing Request) process, which is error-prone and time-consuming. We employ a configuration-driven approach that ensures consistency and meets strict identity requirements such as the inclusion of specific serial numbers mandated by regulatory frameworks. This automation is particularly valuable in production environments where certificates must be generated frequently with varying organizational data but consistent formatting and structure.

The Configuration File Structure

To automate the Distinguished Name (DN) fields that identify the certificate subject, a specifically structured configuration file is utilized. The most critical directive in this configuration is **prompt = no**, which is essential for headless execution via 4D System Workers. Without this directive, OpenSSL would pause execution and wait for interactive terminal input, which is not viable in the context of a SystemWorker running in a background process. By setting **prompt = no**, the configuration instructs OpenSSL to use the values specified in the configuration file without any user interaction, enabling fully automated certificate generation.

```
[ req ]
default_bits      = 2048
default_md        = sha256
distinguished_name = req_distinguished_name
prompt           = no

[ req_distinguished_name ]
C               = US
ST              = California
L               = San Francisco
O               = My Company Inc.
OU              = IT Department
CN              = www.mycompany.com
serialNumber    = SRL-12-345678966
```

The **serialNumber** field in the Distinguished Name deserves special attention as it is often a requirement for government-regulated e-invoicing systems or corporate identity verification schemes. Different jurisdictions may require specific formats for this field, such as VAT numbers in European Union countries, tax identification numbers in other regions, or internal device identifiers in corporate environments.

Cryptographic Resource Generation

This technical note focuses on the generation of various cryptographic assets utilized in different contexts within software applications. Whether supporting encrypted data exchange between business partners, securing web server communications with SSL/TLS, or

implementing digital signatures for document authentication, cryptographic assets form the foundation of modern secure applications. The generation workflow follows a logical progression from the most fundamental element; the private key. Sophisticated derived cryptographic assets build upon this foundation.

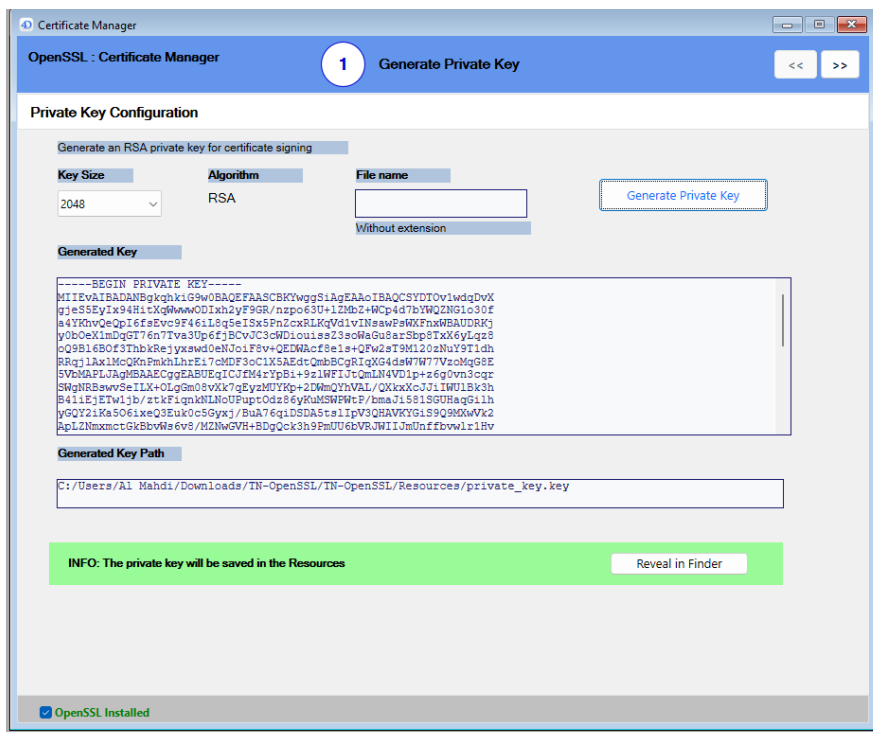
Private Key Generation

The workflow begins with the Private Key, which serves as the "root of trust" for all subsequent cryptographic operations. The private key is the most sensitive element in the entire infrastructure, as possession of this key enables the holder to impersonate the identity associated with certificates derived from it and to decrypt messages encrypted with the corresponding public key. For this integration, we utilize the RSA algorithm with a 2048-bit modulus, which currently provides an optimal balance between high security and computational performance for typical 4D applications. While 4096-bit keys offer even greater security margins, they require significantly more processing power for both generation and use, making 2048-bit keys the preferred choice for most applications.

The OpenSSL command for generating an RSA private key is straight-forward:

```
genrsa -out "private_key.key" 2048
```

The **genrsa** subcommand instructs OpenSSL to generate an RSA private key, the **-out** parameter specifies the output file path where the key will be written, and the final argument 2048 indicates the key size in bits. When this command executes, OpenSSL performs the mathematical operations necessary to create an RSA key pair, writing the private component to the specified file.



The asynchronous execution pattern means that the 4D method returns immediately after initiating the key generation, with the completion callback being invoked once OpenSSL has finished creating and writing the key file. For demonstration purposes, the key is generated with a synchronous version of the method. However, the asynchronous version of every method is available in the demonstration application.

Certificate Signing-Request Generation

The Certificate Signing Request represents the next step in the cryptographic asset generation workflow. A CSR is a formal request submitted to a Certificate Authority (CA) requesting the issuance of a signed certificate, or it can be used for self-signing in development and testing environments. The CSR contains the public key extracted from the previously generated private key along with identity information that will be embedded in the final certificate. This identity information follows the X.509 standard structure and includes fields like Country (C), State/Province (ST), Locality (L), Organization (O), Organizational Unit (OU), and Common Name (CN).

By using the **-config** flag to reference the automated openssl.cnf file, we insert the Serial Number and all Distinguished Name fields automatically, without requiring interactive terminal input or hardcoding values directly in 4D methods. This configuration-driven approach provides significant advantages: it separates identity data from application logic, makes the system easier to maintain when organizational information changes, and enables dynamic generation of CSRs by programmatically creating different configuration files.

The complete OpenSSL command for CSR generation is:

```
req -new -key "private_key.key" -out "request.csr" -config "openssl.cnf"
```

Command Parameters Explained

Parameter	Description
-new	Generates a new certificate request
-key	Specifies the private key file to use
-out	Output filename for the CSR
-config	Path to the OpenSSL configuration file

Self-Signed Certificate Generation

For internal testing, development environments, or private data exchange scenarios where a trusted certificate authority is not required, a self-signed Certificate provides a practical solution. This type of certificate links the private key to the identity defined in the CSR or configuration file, creating a valid X.509 certificate that can be used for encryption and digital signatures. The term "self-signed" indicates that the certificate is signed using the

same private key that it certifies, rather than being signed by a separate Certificate Authority's key. While self-signed certificates are not suitable for public-facing websites due to browser trust warnings, they are perfectly acceptable for internal applications and testing environments.

The OpenSSL command for generating a self-signed certificate combines elements of CSR generation with certificate signing in a single operation:

```
req -new -x509 -key "private_key.key" -config "openssl.cnf" -out "certificate.crt" -days 365
```

The **-x509** flag is the key differentiator here, instructing OpenSSL to output a self-signed certificate instead of a certificate signing request. The **-days** parameter specifies the validity period of the certificate in days, after which the certificate will be considered expired and applications will refuse to trust it.

Signing XML file Generation

The final step in the cryptographic pipeline involves applying a digital signature to an XML document, a requirement that appears frequently in financial workflows. Many e-invoicing systems, purchase order systems, and supply chain management platforms require XML documents to be digitally signed to ensure authenticity and integrity. This is achieved using the S/MIME (Secure/Multipurpose Internet Mail Extensions) standard, which wraps the XML data in a PKCS#7 signature container. This signature format enables recipients to verify both the sender's identity (by validating the certificate) and the data's integrity (by verifying that the content hasn't been modified since signing).

Using the `smime` command in OpenSSL, the 4D application performs a binary sign operation that specifically uses a password-protected private key to generate a .p7s output in PEM format. The command construction involves multiple critical parameters that must be correctly specified for the signature to be valid and acceptable to receiving systems.

```
$cmd="smime -sign -binary -nodetach"  
$cmd+=" -in "+Char(34)+$pathXMLIN+Char(34)  
$cmd+=" -out "+Char(34)+$pathXMLOUT+Char(34)  
$cmd+=" -signer "+Char(34)+$pathCert+Char(34)  
$cmd+=" -inkey "+Char(34)+$pathKey+Char(34)  
$cmd+=" -passin pass:"+$password  
$cmd+=" -outform PEM"
```

Several flags in this command need detailed explanation due to their critical importance for signature validity. The **-binary** flag instructs OpenSSL to treat the input file as binary data rather than text, which prevents any line-ending conversions that could invalidate the signature. XML files, despite being text-based, should always be signed in binary mode to preserve exact byte sequences including line endings and whitespace, as even minor modifications to these elements will cause signature verification to fail. The **-nodetach** flag specifies that the original XML content should be included within the output file alongside the signature, creating a self-contained signed message bundle. This is the standard approach for most document exchange scenarios, as it allows recipients to access both

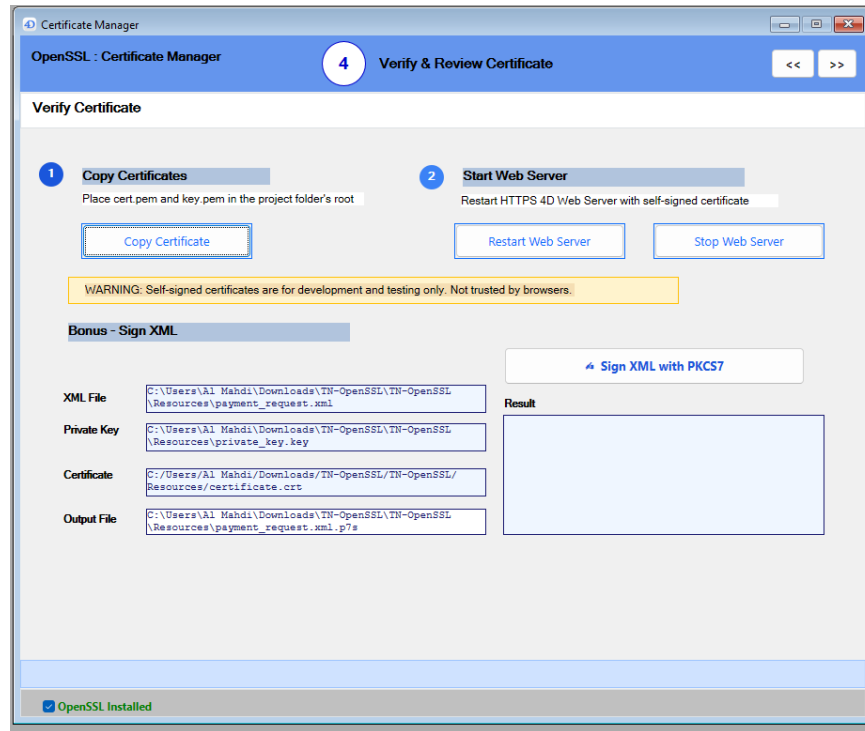
the original content and the signature in a single file. The **-passin pass:** option provides the password for the encrypted private key, with the actual password appended immediately after the colon. Finally, **-outform PEM** ensures that the output is in PEM (Privacy-Enhanced Mail) format, which is a base64-encoded representation that is widely compatible with email systems and document management platforms.

4D Web Server Integration

Securing a 4D Web Server with SSL/TLS certificates generated through this OpenSSL integration requires copying the certificate and private key files to specific locations where 4D expects to find them. When 4D's web server starts with SSL/TLS enabled, it automatically searches for files named `cert.pem` and `key.pem` in the project folder's root directory. By renaming and copying the generated `.pem` and `.key` files to these standard names and locations, developers can seamlessly integrate OpenSSL-generated certificates with 4D's built-in web server.

The implementation involves using 4D's File and Folder objects to locate source files in the Resources folder and destination locations in the project folder, then using the `.copyTo()` method with the `fk overwrite` option to ensure that existing certificates are replaced with newly generated ones. This copying step might be triggered by a button in the user interface, allowing administrators to update certificates without restarting the application or manually manipulating files in the file system.

Once the certificate files are in place, the web server is configured and started programmatically using the `4D.WebServer` class, which provides control over both HTTP and HTTPS settings. The implementation first stops any currently running server instance to ensure a clean restart with the new certificate configuration, then constructs a settings object that explicitly enables both HTTP on port 80 and HTTPS on port 443 before calling the `start()` method with these settings.



Best Practices & Security Considerations

Certificate Management

Proper certificate management extends far beyond the technical implementation of generation and storage, encompassing comprehensive lifecycle management practices that ensure security throughout the entire lifespan of cryptographic assets.

Private keys must be stored in secure locations with appropriate file permissions that restrict access to only the minimal set of users and processes that require it. On Unix-like systems including macOS and Linux, this typically means setting file permissions to 600 (readable and writable only by the owner) using the **chmod** command immediately after key generation. Windows systems should use NTFS permissions to similarly restrict access to the file owner only.

An absolute requirement that cannot be overemphasized is that private keys should never be committed to version control systems such as Git. Even in private repositories, storing private keys in version control creates unacceptable security risks because version control systems are designed to preserve all historical versions of files, meaning that a key accidentally committed once will remain in the repository history even after being deleted from the current version. Many high-profile security breaches have resulted from private keys or API credentials accidentally committed to public repositories on GitHub or similar platforms. Developers should use .gitignore files or equivalent mechanisms to ensure that files with .key, .pem, or similar extensions are automatically excluded from version control.

Production Deployment

Production deployment of cryptographic systems requires adherence to industry standards and best practices that may differ significantly from development and testing approaches. For public-facing services accessible over the internet, self-signed certificates are unacceptable due to browser security warnings that create poor user experience. Instead, production systems should use certificates issued by trusted Certificate Authorities (CA) that are included in browser and operating system trust stores. Let's Encrypt provides free automated certificates suitable for many scenarios, while commercial CAs offer extended validation certificates and additional support services.

Troubleshooting Common Issues

OpenSSL Not Found

The most fundamental issue that can occur is the inability to locate the OpenSSL executable, manifesting as an error message indicating that the system cannot find the specified command. This problem typically occurs on newly configured systems where OpenSSL has not been installed, or on systems where OpenSSL is installed in a non-standard location that differs from the path specified in the application configuration. Troubleshooting begins with verifying that OpenSSL is installed by opening a terminal or command prompt and executing **openssl version**, which should display version information if OpenSSL is properly installed and accessible via the system PATH.

In the 4D application, users can choose a custom OpenSSL folder location and point to an executable OpenSSL file. The application will then use this file. This is to say that for standard usage of the demonstration application, users do not require adding OpenSSL to system variables.

Invalid XML Signature

First, verify that the **-binary** flag is included in the OpenSSL **smime** command. Without this flag, OpenSSL may perform line-ending conversions on the XML data, changing `\n` to `\r\n` or vice versa depending on the platform. Since digital signatures are computed over the exact byte sequence of the data, any modification to line endings invalidates the signature even though the XML remains functionally equivalent and well-formed. The **-binary** flag instructs OpenSSL to preserve the exact byte sequence without any transformation.

Conclusion

This integration of OpenSSL with 4D's asynchronous SystemWorker class provides a robust solution for implementing cryptographic operations within 4D applications. The callback-based architecture ensures responsive applications that remain usable during potentially lengthy cryptographic operations, while maintaining the full power and flexibility of OpenSSL's command-line interface without the limitations that sometimes affect higher-level technologies. By following the patterns and practices outlined in this document, developers can confidently implement complex cryptographic workflows including generating private keys, certificate signing requests, self-signed certificates, and signed XML documents.

The key achievements of this implementation include true asynchronous execution with non-blocking user interface behavior, robust error handling through a comprehensive callback system that captures and reports errors at multiple stages of operation, cross-platform compatibility through automatic path handling and platform detection that works seamlessly on both Windows and macOS. Logging capabilities that create audit trails for security compliance and troubleshooting are also key elements. The automatic worker detection and self-spawning mechanism abstracts away the complexity of the SystemWorker callback requirements, making asynchronous operations accessible to developers regardless of the familiarity with the underlying technical details.